Mini project report on

GIGSY: AI-POWERED FREELANCE MARKETPLACE

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE23CS351A – DBMS Project

Submitted by:

Vikas Kulakarni	**PES2UG24CS922**
Shrinivas Shrikant Jainapur	**PES2UG24CS494**
Shivangouda	**PES2UG24CS471**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Gigsy: AI-Powered Freelance Marketplace

is a bonafide work carried out by

Vikas Kulakarni	**PES2UG24CS922**
Shrinivas Shrikant Jainapur	**PES2UG24CS494**
Shivangouda	**PES2UG24CS471**

In partial fulfilment for the completion of fifth semester DBMS Project (UE23CS351A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Gigsy: AI-Powered Freelance Marketplace** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Vikas Kulakarni	**PES2UG24CS922**
Shrinivas Shrikant Jainapur	**PES2UG24CS494**
Shivangouda	**PES2UG24CS471**

ACKNOWLEDGEMENT

We would like to express our gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS351A - DBMS Project.

We take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to us various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from our family and friends.

ABSTRACT

Gigsy is an AI-powered freelance marketplace designed to connect clients with suitable freelancers through gig posting, proposals, contract management, milestones, messaging, notifications, reviews, and semantic skill matching. The application uses Next.js and TypeScript for the frontend and API layer, MySQL as the relational database, and a FastAPI service integrated with ChromaDB for vector-based semantic search. The database design includes users, client and freelancer profiles, gigs, proposals, skills, contracts, milestones, payments, reviews, messages, and notifications. Advanced DBMS concepts are demonstrated through relational constraints, views, a stored procedure, a trigger, aggregate queries, and transaction-based workflow operations. Verification included successful execution of the matching procedure, dynamic trigger testing with rollback, and reconciliation of a contract amount against its three milestones.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	10
2.	PROBLEM DEFINITION	12
3.	ER MODEL	13
4.	ER TO RELATIONAL MAPPING	15
5.	DDL STATEMENTS	17
6.	DML STATEMENTS	20
7.	QUERIES (SIMPLE QUERY AND UPDATE AND DELETE OPERATION, CORRELATED QUERY AND NESTED QUERY)	21
8.	STORED PROCEDURE, FUNCTIONS AND TRIGGERS	25
9.	FRONT END DEVELOPMENT	28
10.	CONCLUSION AND TEAM CONTRIBUTIONS	35
11.	REQUIREMENT COVERAGE AND FINALIZATION STATUS	37
REFERENCES/BIBLIOGRAPHY		39
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		40

LIST OF TABLES

Table No.	Title	Page No.
1.1	Technology stack used in Gigsy	10
4.1	Complete relational mapping of all 17 base tables and declared key relationships	15
9.1	Test cases and available verification evidence	33
10.1	Team members, responsibilities and contributions	35
11.1	Faculty requirement coverage and remaining actions	37

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	ER Diagram	**14**
5.1	SHOW TABLES Output	**17**
5.2	DESCRIBE Output	**18**
5.3	View Outputs	**18**
6.1	INSERT and Sample Data	**20**
7.1	SELECT with WHERE and ORDER BY	**21**
7.2	JOIN Query	**21**
7.3	Aggregate Functions	**21**
7.4	GROUP BY	**22**
7.5	GROUP BY with HAVING	**22**
7.6	UPDATE Operation	**23**
7.7	DELETE Operation	**23**
7.8	Correlated Query	**24**
7.9	Nested Query	**24**
8.1	Stored Procedure	**25**
8.2	Trigger Definition	**25**
8.3	CREATE TRIGGER	**26**
8.4	Trigger Test and ROLLBACK	**26**
8.5	Transaction Verification	**27**
9.1	Landing Page	**28**
9.2	Freelancer Dashboard	**29**
9.3	Client Dashboard	**29**

9.4 Create Gig Form **30**

9.5 Explore Gigs **30**

9.6 Proposal Review **31**

9.7 Contract and Milestones **31**

9.8 Semantic Search Results **32**

1. INTRODUCTION

1.1 BACKGROUND

Online freelance marketplaces help clients access independent professionals for short-term and project-based work. As the number of gigs and freelancer profiles grows, manually identifying relevant talent becomes difficult.

1.2 PROJECT OVERVIEW

Gigsy provides a two-role marketplace for clients and freelancers. Clients publish gigs and review proposals; freelancers maintain profiles, list skills, and apply to suitable opportunities. The platform supports proposal acceptance, contract creation, milestone tracking, messaging, notifications, reviews, and semantic matching.

1.3 OBJECTIVES

The objectives are to design a relational database, implement core marketplace workflows, enforce data integrity using keys and constraints, demonstrate advanced SQL features, and integrate semantic matching using vector similarity search.

1.4 TECHNOLOGY STACK

Table 1.1: Technology stack used in Gigsy.

Layer	Technology
Frontend and API	Next.js 15, React 19, TypeScript
Relational database	MySQL 8
Vector search	ChromaDB
Semantic service	FastAPI and sentence-transformers/all-MiniLM-L6-v2
Development and validation	Node.js, npm, Python, and Git

1.5 SCOPE

The project covers user roles, freelancer profiles and skills, gig listings, proposals, hiring workflow, contracts, milestones, messages, notifications, reviews, and semantic search. The database contains

legacy marketplace relations as well as the current gigsy_* hiring workflow tables; these are documented separately.

2. PROBLEM DEFINITION

2.1 PROBLEM STATEMENT

Clients may find it difficult to identify freelancers whose skills and experience match a gig description. A marketplace also needs reliable storage and transaction handling for proposals, contracts, milestones, messages, and reviews.

2.2 EXISTING CHALLENGES

Keyword-only search may miss semantically related skills. Marketplace workflows can create inconsistent records if proposal acceptance, contract creation, milestone generation, and gig status changes are not handled together. Unconstrained data entry can also cause invalid references and inconsistent aggregates.

2.3 PROPOSED SOLUTION

Gigsy combines a MySQL-backed marketplace with a semantic search service. MySQL remains the source of truth for users, gigs, proposals, and contracts. ChromaDB stores gig embeddings and returns semantically similar gig identifiers; the application resolves those identifiers against current MySQL records.

2.4 FUNCTIONAL REQUIREMENTS

The system supports role-based registration and login, freelancer profile and skill management, gig creation and browsing, proposal submission and acceptance, contract and milestone creation, messaging, notifications, reviews, and semantic matching.

2.5 NON-FUNCTIONAL REQUIREMENTS

The application aims for data consistency, referential integrity, maintainability, responsive interaction, separation of services, and secure handling of service credentials.

3. ER MODEL

3.1 MAIN ENTITIES

The principal entities include users, client, freelancer, skill, freelancer_skill, gig, gig_skill, proposal, contract, order_milestone, payment, review, message, and notification. The current application workflow additionally uses gigsy_contracts, gigsy_milestones, and gigsy_notifications.

3.2 RELATIONSHIPS AND CARDINALITIES

A user may have a client or freelancer profile. A client can create multiple gigs. A gig can receive multiple proposals. Freelancers and skills have a many-to-many relationship through freelancer_skill; gigs and skills have a many-to-many relationship through gig_skill. Contracts connect accepted proposals to clients and freelancers. Contracts can contain multiple milestones.

3.3 ER DIAGRAM

The ER diagram below was generated from the Gigsy MySQL schema export. It presents the 17 base tables and their 23 declared foreign-key relationships. PK denotes a primary key and FK denotes a foreign key. The legacy and current application workflow tables are shown in separate groups.

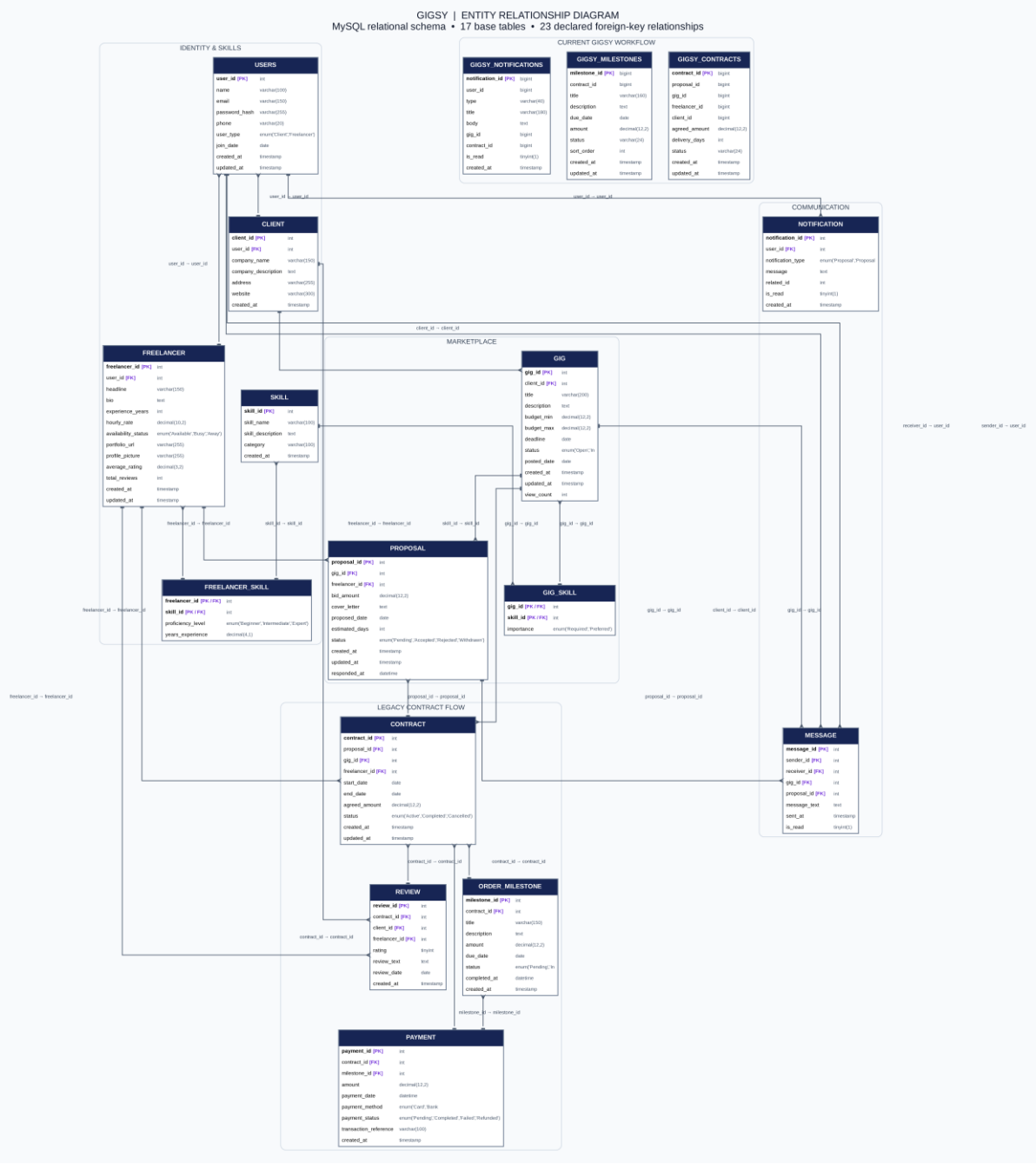


Figure 3.1: ER Diagram

4. ER TO RELATIONAL MAPPING

4.1 STEPS OF ALGORITHM FOR CHOSEN PROBLEM

The ER model is converted to relations using the following steps:

1. Identify entity sets and their primary keys.
2. Convert each strong entity into a relation.
3. Represent one-to-many relationships using foreign keys on the many side.
4. Represent many-to-many relationships using associative relations.
5. Map relationship attributes into the corresponding relation.
6. Define primary-key, unique, check, and foreign-key constraints.

4.2 COMPLETE DIAGRAM OF RELATIONAL MAPPING

Table 4.1 lists all 17 base relations in the supplied MySQL 8.0.46 structure export. It distinguishes declared foreign keys from logical links maintained by application code. The export contains 23 declared foreign-key constraints; the current gigsy_* workflow tables do not declare foreign keys.

Table 4.1: Complete relational mapping of all 17 base tables and declared key relationships.

Relation	Primary Key	Declared Foreign Key(s)
client	client_id	user_id → users.user_id
contract	contract_id	proposal_id → proposal.proposal_id gig_id → gig.gig_id freelancer_id → freelancer.freelancer_id
freelancer	freelancer_id	user_id → users.user_id
freelancer_skill	(freelancer_id, skill_id)	freelancer_id → freelancer.freelancer_id skill_id → skill.skill_id
gig	gig_id	client_id → client.client_id
gig_skill	(gig_id, skill_id)	gig_id → gig.gig_id skill_id → skill.skill_id
gigsy_contracts	contract_id	None declared; proposal_id and gig_id are UNIQUE logical links
gigsy_milestones	milestone_id	None declared; contract_id is an indexed logical link
gigsy_notifications	notification_id	None declared; user_id/gig_id/contract_id are logical references

Relation	Primary Key	Declared Foreign Key(s)
message	message_id	sender_id → users.user_id receiver_id → users.user_id gig_id → gig.gig_id proposal_id → proposal.proposal_id
notification	notification_id	user_id → users.user_id
order_milestone	milestone_id	contract_id → contract.contract_id
payment	payment_id	contract_id → contract.contract_id milestone_id → order_milestone.milestone_id
proposal	proposal_id	gig_id → gig.gig_id freelancer_id → freelancer.freelancer_id
review	review_id	contract_id → contract.contract_id client_id → client.client_id freelancer_id → freelancer.freelancer_id
skill	skill_id	None
users	user_id	None

The schema also defines three views (freelancer_profile_view, freelancer_skills_view and open_gigs_view), the MatchFreelancersBySkills stored procedure, and the trg_update_freelancer_rating trigger. The three gigsy_* workflow tables are part of the current application model but their references are application-enforced, not database-enforced, in this export.

The relational mapping is summarized in Table 4.1. The schema contains 17 base relations. Declared foreign keys enforce the relationships listed in the table; references in the current gigsy_* workflow tables are maintained by the application and are not declared as foreign keys in the supplied schema export.

5. DDL STATEMENTS

STATEMENTS WITH SCREEN SHOTS OF THE TABLE CREATION

5.1 Database Objects

The verified database contains 17 base tables, 3 views, 1 stored procedure, and 1 trigger. The base tables are client, contract, freelancer, freelancer_skill, gig, gig_skill, gigsy_contracts, gigsy_milestones, gigsy_notifications, message, notification, order_milestone, payment, proposal, review, skill, and users.

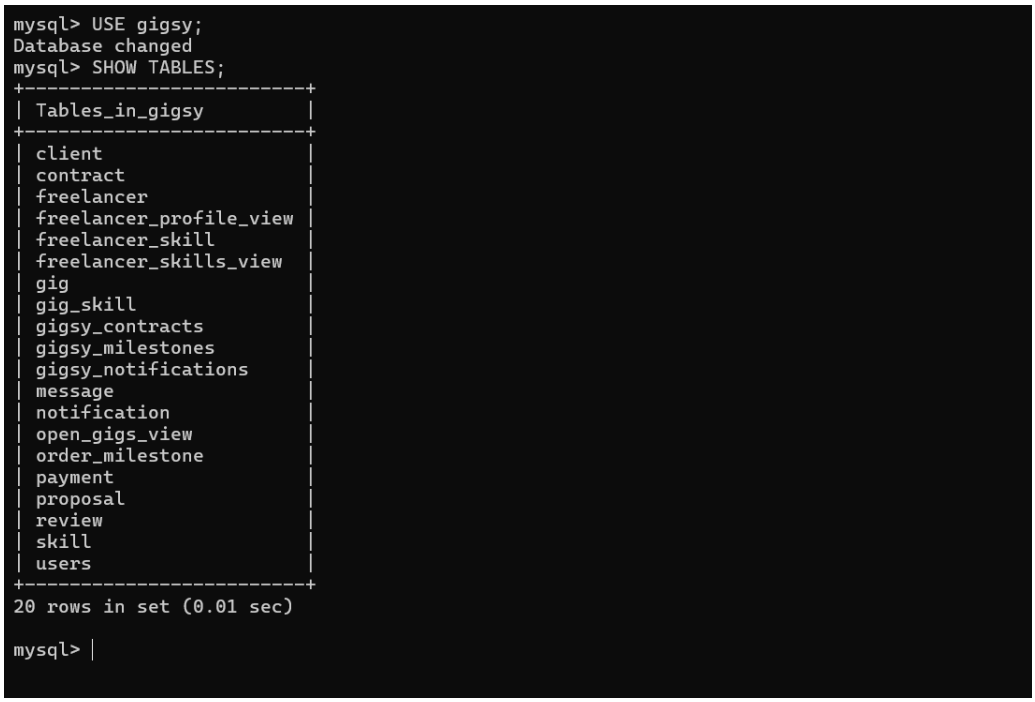


Figure 5.1: SHOW TABLES Output

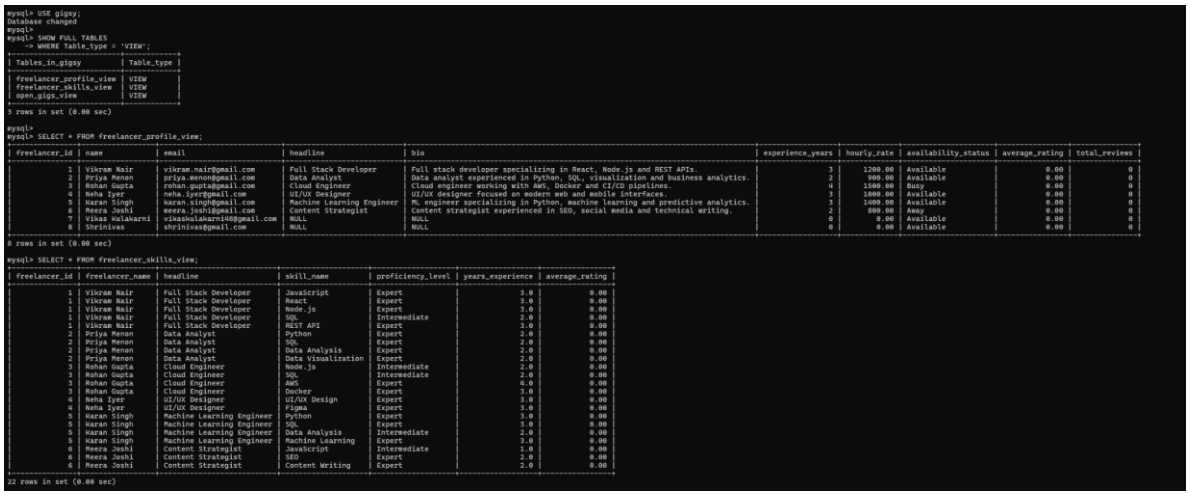
```
mysql> DESCRIBE users;
+---------------+---------------------------------+------+-----+-------------------+-----------------------------------------------+
| Field         | Type                            | Null | Key | Default           | Extra                                         |
+---------------+---------------------------------+------+-----+-------------------+-----------------------------------------------+
| user_id       | int                             | NO   | PRI | NULL              | auto_increment                                |
| name          | varchar(100)                    | NO   |     | NULL              |                                               |
| email         | varchar(150)                    | NO   | UNI | NULL              |                                               |
| password_hash | varchar(255)                    | NO   |     | NULL              |                                               |
| phone         | varchar(20)                     | YES  |     | NULL              |                                               |
| user_type     | enum('Client','Freelancer')     | NO   |     | NULL              |                                               |
| join_date     | date                            | NO   |     | curdate()         | DEFAULT_GENERATED                             |
| created_at    | timestamp                       | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED                             |
| updated_at    | timestamp                       | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+---------------+---------------------------------+------+-----+-------------------+-----------------------------------------------+
9 rows in set (0.00 sec)

mysql> DESCRIBE gig;
+-------------+-----------------------------------------------------+------+-----+-------------------+-----------------------------------------------+
| Field       | Type                                                | Null | Key | Default           | Extra                                         |
+-------------+-----------------------------------------------------+------+-----+-------------------+-----------------------------------------------+
| gig_id      | int                                                 | NO   | PRI | NULL              | auto_increment                                |
| client_id   | int                                                 | NO   | MUL | NULL              |                                               |
| title       | varchar(200)                                        | NO   | MUL | NULL              |                                               |
| description | text                                                | NO   |     | NULL              |                                               |
| budget_min  | decimal(12,2)                                       | YES  |     | NULL              |                                               |
| budget_max  | decimal(12,2)                                       | YES  |     | NULL              |                                               |
| deadline    | date                                                | YES  | MUL | NULL              |                                               |
| status      | enum('Open','In Progress','Completed','Cancelled')  | YES  | MUL | Open              |                                               |
| posted_date | date                                                | NO   |     | curdate()         | DEFAULT_GENERATED                             |
| created_at  | timestamp                                           | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED                             |
| updated_at  | timestamp                                           | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
| view_count  | int                                                 | NO   |     | 0                 |                                               |
+-------------+-----------------------------------------------------+------+-----+-------------------+-----------------------------------------------+
12 rows in set (0.00 sec)

mysql> DESCRIBE proposal;
+-----------------+-----------------------------------------------------+------+-----+-------------------+-----------------------------------------------+
| Field           | Type                                                | Null | Key | Default           | Extra                                         |
+-----------------+-----------------------------------------------------+------+-----+-------------------+-----------------------------------------------+
| proposal_id     | int                                                 | NO   | PRI | NULL              | auto_increment                                |
| gig_id          | int                                                 | NO   | MUL | NULL              |                                               |
| freelancer_id   | int                                                 | NO   | MUL | NULL              |                                               |
| bid_amount      | decimal(12,2)                                       | NO   |     | NULL              |                                               |
| cover_letter    | text                                                | YES  |     | NULL              |                                               |
| proposed_date   | date                                                | NO   |     | curdate()         | DEFAULT_GENERATED                             |
| estimated_days  | int                                                 | YES  |     | NULL              |                                               |
| status          | enum('Pending','Accepted','Rejected','Withdrawn')   | YES  | MUL | Pending           |                                               |
| created_at      | timestamp                                           | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED                             |
| updated_at      | timestamp                                           | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
| responded_at    | datetime                                            | YES  |     | NULL              |                                               |
+-----------------+-----------------------------------------------------+------+-----+-------------------+-----------------------------------------------+
11 rows in set (0.00 sec)

mysql>
```

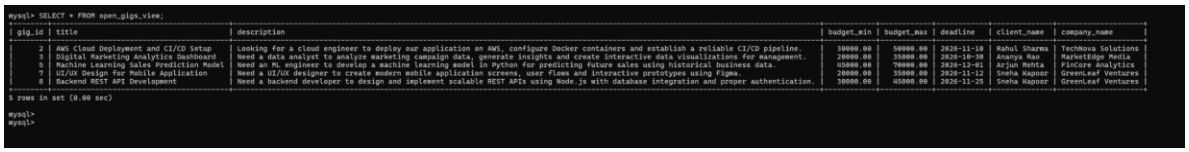
Figure 5.2: DESCRIBE Output



```
mysql> USE gigsy;
Database changed
mysql>
mysql> SHOW FULL TABLES
    -> WHERE Table_type = 'VIEW';
+-----------------------+------------+
| Tables_in_gigsy       | Table_type |
+-----------------------+------------+
| freelancer_profile_view | VIEW     |
| freelancer_skills_view  | VIEW     |
| open_gigs_view          | VIEW     |
+-----------------------+------------+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM freelancer_profile_view;
+---------------+-------------+-------------------------+-------------------------+-----------------------------------------------------------------------------+-----------------+-------------+-------------------+----------------+---------------+
| freelancer_id | name        | email                   | headline                | bio                                                                         | experience_years | hourly_rate | availability_status | average_rating | total_reviews |
+---------------+-------------+-------------------------+-------------------------+-----------------------------------------------------------------------------+-----------------+-------------+-------------------+----------------+---------------+
|             1 | Vikram Nair | vikram.nair@gmail.com   | Full Stack Developer    | Full stack developer specializing in React, Node.js and REST APIs.          |               3 |     1200.00 | Available         |           0.00 |             0 |
|             2 | Priya Menon | priya.menon@gmail.com   | Data Analyst            | Data analyst experienced in Python, SQL, visualization and business analytics. |               2 |      900.00 | Available         |           0.00 |             0 |
|             3 | Rohan Gupta | rohan.gupta@gmail.com   | Cloud Engineer          | Cloud engineer working with AWS, Docker and CI/CD pipelines.                |               4 |     1500.00 | Busy              |           0.00 |             0 |
|             4 | Neha Iyer   | neha.iyer@gmail.com     | UI/UX Designer          | UI/UX designer focused on modern web and mobile interfaces.                 |               3 |     1000.00 | Available         |           0.00 |             0 |
|             5 | Karan Singh | karan.singh@gmail.com   | Machine Learning Engineer | ML engineer specializing in Python, machine learning and predictive analytics. |               3 |     1600.00 | Available         |           0.00 |             0 |
|             6 | Meera Joshi | meera.joshi@gmail.com   | Content Strategist      | Content strategist experienced in SEO, social media and technical writing.  |               2 |      800.00 | Away              |           0.00 |             0 |
|             7 | Vikas Kalakarni | vikaskalakarni48@gmail.com | NULL                 | NULL                                                                        |               0 |        0.00 | Available         |           0.00 |             0 |
|             8 | Shrinivas   | shrinivas@gmail.com     | NULL                    | NULL                                                                        |               0 |        0.00 | Available         |           0.00 |             0 |
+---------------+-------------+-------------------------+-------------------------+-----------------------------------------------------------------------------+-----------------+-------------+-------------------+----------------+---------------+
8 rows in set (0.00 sec)

mysql> SELECT * FROM freelancer_skills_view;
+---------------+-----------------+-------------------------+-----------------+-------------------+------------------+----------------+
| freelancer_id | freelancer_name | headline                | skill_name      | proficiency_level | years_experience | average_rating |
+---------------+-----------------+-------------------------+-----------------+-------------------+------------------+----------------+
|             1 | Vikram Nair     | Full Stack Developer    | JavaScript      | Expert            |              3.0 |           0.00 |
|             1 | Vikram Nair     | Full Stack Developer    | React           | Expert            |              3.0 |           0.00 |
|             1 | Vikram Nair     | Full Stack Developer    | Node.js         | Expert            |              3.0 |           0.00 |
|             1 | Vikram Nair     | Full Stack Developer    | SQL             | Intermediate      |              2.0 |           0.00 |
|             1 | Vikram Nair     | Full Stack Developer    | REST API        | Expert            |              3.0 |           0.00 |
|             2 | Priya Menon     | Data Analyst            | Python          | Expert            |              2.0 |           0.00 |
|             2 | Priya Menon     | Data Analyst            | SQL             | Expert            |              2.0 |           0.00 |
|             2 | Priya Menon     | Data Analyst            | Data Analysis   | Expert            |              2.0 |           0.00 |
|             2 | Priya Menon     | Data Analyst            | Data Visualization | Expert         |              2.0 |           0.00 |
|             3 | Rohan Gupta     | Cloud Engineer          | Node.js         | Intermediate      |              2.0 |           0.00 |
|             3 | Rohan Gupta     | Cloud Engineer          | SQL             | Intermediate      |              2.0 |           0.00 |
|             3 | Rohan Gupta     | Cloud Engineer          | AWS             | Expert            |              4.0 |           0.00 |
|             3 | Rohan Gupta     | Cloud Engineer          | Docker          | Expert            |              3.0 |           0.00 |
|             4 | Neha Iyer       | UI/UX Designer          | UI/UX Design    | Expert            |              3.0 |           0.00 |
|             4 | Neha Iyer       | UI/UX Designer          | Figma           | Expert            |              3.0 |           0.00 |
|             5 | Karan Singh     | Machine Learning Engineer | Python        | Expert            |              3.0 |           0.00 |
|             5 | Karan Singh     | Machine Learning Engineer | SQL           | Expert            |              3.0 |           0.00 |
|             5 | Karan Singh     | Machine Learning Engineer | Data Analysis | Intermediate      |              2.0 |           0.00 |
|             5 | Karan Singh     | Machine Learning Engineer | Machine Learning | Expert         |              3.0 |           0.00 |
|             6 | Meera Joshi     | Content Strategist      | JavaScript      | Intermediate      |              1.0 |           0.00 |
|             6 | Meera Joshi     | Content Strategist      | SEO             | Expert            |              2.0 |           0.00 |
|             6 | Meera Joshi     | Content Strategist      | Content Writing | Expert            |              2.0 |           0.00 |
+---------------+-----------------+-------------------------+-----------------+-------------------+------------------+----------------+
22 rows in set (0.00 sec)
```



```
mysql> SELECT * FROM open_gigs_view;
+--------+-----------------------------------+-----------------------------------------------------------------------------------------------------------------------------------+------------+------------+------------+-------------+-----------------+
| gig_id | title                             | description                                                                                                                       | budget_min | budget_max | deadline   | client_name | company_name    |
+--------+-----------------------------------+-----------------------------------------------------------------------------------------------------------------------------------+------------+------------+------------+-------------+-----------------+
|      2 | AWS Cloud Deployment and CI/CD Setup | Looking for a cloud engineer to deploy our application on AWS, configure Docker containers and establish a reliable CI/CD pipeline. |   30000.00 |   50000.00 | 2026-11-10 | Rahul Sharma | TechNova Solutions |
|      3 | Digital Marketing Analytics Dashboard | Need a data analyst to analyze marketing campaign data, generate insights and create interactive data visualizations for management. |   20000.00 |   35000.00 | 2026-10-30 | Ananya Rao  | MarketEdge Media |
|      5 | Machine Learning Sales Prediction Model | Need an ML engineer to develop a machine learning model in Python for predicting future sales using historical business data.     |   60000.00 |   90000.00 | 2026-12-01 | Arjun Mehta | FinCore Analytics |
|      7 | UI/UX Design for Mobile Application | Need a UI/UX designer to create modern mobile application screens, user flows and interactive prototypes using Figma.             |   20000.00 |   35000.00 | 2026-11-12 | Sneha Kapoor | GreenLeaf Ventures |
|      8 | Backend REST API Development      | Need a backend developer to design and implement scalable REST APIs using Node.js with database integration and proper authentication. |   30000.00 |   45000.00 | 2026-11-25 | Sneha Kapoor | GreenLeaf Ventures |
+--------+-----------------------------------+-----------------------------------------------------------------------------------------------------------------------------------+------------+------------+------------+-------------+-----------------+
5 rows in set (0.00 sec)

mysql>
mysql>
```

Figure 5.3: View Outputs

5.2 Key Constraints

The schema uses primary keys for entity identity, foreign keys for relationships, unique constraints to prevent duplicate review-per-contract and other duplicate associations, and a CHECK constraint

restricting review ratings to values from 1 through 5. The review table references contract, client, and freelancer.

5.3 Representative DDL

The integrated MySQL schema dump contains CREATE TABLE definitions for the current Gigsy entities, primary and unique keys, indexes, foreign-key constraints and CHECK constraints. The dump also contains DROP TABLE IF EXISTS statements for controlled schema rebuilds; it is a schema export and must not be imported into a populated submission database without a backup.

Figures 5.1–5.3 are original MySQL execution screenshots shared during the Gigsy project work. The integrated schema is documented in this chapter.

6. DML STATEMENTS

STATEMENTS WITH SCREEN SHOTS OF THE TABLE WITH INSERTED VALUES

6.1 INSERT Operation

```
mysql> SELECT * FROM users;
+---------+---------------+-----------------------+---------------+------------+-------------+---------------------+---------------------+---------------------+
| user_id | name          | email                 | password_hash | phone      | user_type   | join_date           | created_at          | updated_at          |
+---------+---------------+-----------------------+---------------+------------+-------------+---------------------+---------------------+---------------------+
|       1 | Rahul Sharma  | rahul.sharma@gmail.com | hashed_pw_001 | 9876543210 | Client      | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
|       2 | Ananya Rao    | ananya.rao@gmail.com   | hashed_pw_002 | 9876543211 | Client      | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
|       3 | Arjun Mehta   | arjun.mehta@gmail.com  | hashed_pw_003 | 9876543212 | Client      | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
|       4 | Sneha Kapoor  | sneha.kapoor@gmail.com | hashed_pw_004 | 9876543213 | Client      | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
|       5 | Vikram Nair   | vikram.nair@gmail.com  | hashed_pw_005 | 9876543220 | Freelancer  | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
|       6 | Priya Menon   | priya.menon@gmail.com  | hashed_pw_006 | 9876543221 | Freelancer  | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
|       7 | Rohan Gupta   | rohan.gupta@gmail.com  | hashed_pw_007 | 9876543222 | Freelancer  | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
|       8 | Neha Iyer     | neha.iyer@gmail.com    | hashed_pw_008 | 9876543223 | Freelancer  | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
|       9 | Karan Singh   | karan.singh@gmail.com  | hashed_pw_009 | 9876543224 | Freelancer  | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
|      10 | Meera Joshi   | meera.joshi@gmail.com  | hashed_pw_010 | 9876543225 | Freelancer  | 2026-10-02 | 2026-10-02 23:47:43 | 2026-10-02 23:47:43 |
+---------+---------------+-----------------------+---------------+------------+-------------+---------------------+---------------------+---------------------+
10 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM gig;
+--------+-----------+-----------------------------------------+-----------------------------------------
```

7. QUERIES

7.1 SIMPLE QUERY WITH GROUP BY, AGGREGATE

```
mysql> SELECT
    ->     gig_id,
    ->     title,
    ->     budget_min,
    ->     budget_max
    -> FROM gig
    -> WHERE status = 'Open'
    -> ORDER BY budget_max DESC;
+--------+-----------------------------------------+------------+------------+
| gig_id | title                                   | budget_min | budget_max |
+--------+-----------------------------------------+------------+------------+
|      5 | Machine Learning Sales Prediction Model |   45000.00 |   70000.00 |
|      2 | AWS Cloud Deployment and CI/CD Setup    |   30000.00 |   50000.00 |
|      8 | Backend REST API Development            |   30000.00 |   45000.00 |
|      3 | Digital Marketing Analytics Dashboard   |   20000.00 |   35000.00 |
|      7 | UI/UX Design for Mobile Application     |   20000.00 |   35000.00 |
+--------+-----------------------------------------+------------+------------+
5 rows in set (0.00 sec)
```

Figure 7.1: SELECT with WHERE and ORDER BY

```
mysql> SELECT
    ->     p.proposal_id,
    ->     g.title AS gig_title,
    ->     u.name AS freelancer_name,
    ->     p.bid_amount,
    ->     p.status
    -> FROM proposal p
    -> JOIN gig g ON p.gig_id = g.gig_id
    -> JOIN freelancer f ON p.freelancer_id = f.freelancer_id
    -> JOIN users u ON f.user_id = u.user_id;
+-------------+-----------------------------------------+-----------------+------------+----------+
| proposal_id | gig_title                               | freelancer_name | bid_amount | status   |
+-------------+-----------------------------------------+-----------------+------------+----------+
|           1 | Build a React and Node.js Web Application | Vikram Nair     |   52000.00 | Pending  |
|           4 | AWS Cloud Deployment and CI/CD Setup    | Vikram Nair     |   45000.00 | Pending  |
|          13 | Backend REST API Development            | Vikram Nair     |   38000.00 | Pending  |
|           5 | Digital Marketing Analytics Dashboard   | Priya Menon     |   28000.00 | Pending  |
|           9 | Machine Learning Sales Prediction Model | Priya Menon     |   52000.00 | Pending  |
|          10 | Business Data Analysis Using Python and SQL | Priya Menon     |   33000.00 | Accepted |
|           2 | Build a React and Node.js Web Application | Rohan Gupta     |   48000.00 | Accepted |
|           3 | AWS Cloud Deployment and CI/CD Setup    | Rohan Gupta     |   42000.00 | Pending  |
|          14 | Backend REST API Development            | Rohan Gupta     |   40000.00 | Pending  |
|          12 | UI/UX Design for Mobile Application     | Neha Iyer       |   28000.00 | Pending  |
|           6 | Digital Marketing Analytics Dashboard   | Karan Singh     |   32000.00 | Pending  |
|           8 | Machine Learning Sales Prediction Model | Karan Singh     |   58000.00 | Pending  |
|          11 | Business Data Analysis Using Python and SQL | Karan Singh     |   36000.00 | Pending  |
|           7 | SEO and Content Strategy                | Meera Joshi     |   19000.00 | Accepted |
+-------------+-----------------------------------------+-----------------+------------+----------+
14 rows in set (0.00 sec)
```

Figure 7.2: JOIN Query

```
mysql> SELECT
    ->     MIN(bid_amount) AS minimum_bid,
    ->     MAX(bid_amount) AS maximum_bid,
    ->     AVG(bid_amount) AS average_bid,
    ->     SUM(bid_amount) AS total_bid_value
    -> FROM proposal;
+-------------+-------------+---------------+-----------------+
| minimum_bid | maximum_bid | average_bid   | total_bid_value |
+-------------+-------------+---------------+-----------------+
|    19000.00 |    58000.00 | 39357.142857 |       551000.00 |
+-------------+-------------+---------------+-----------------+
1 row in set (0.00 sec)
```

Figure 7.3: Aggregate Functions

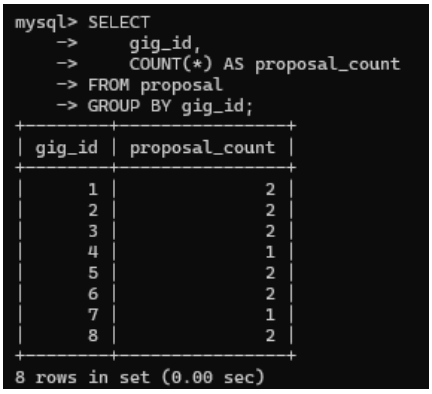


Figure 7.4: GROUP BY

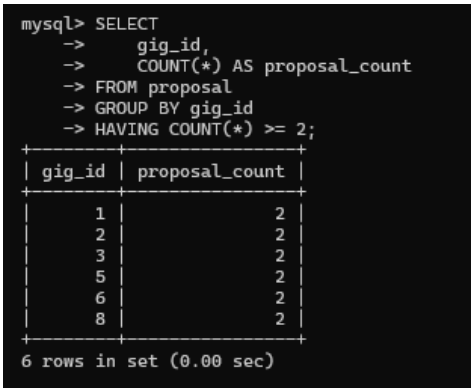


Figure 7.5: GROUP BY with HAVING

Evidence note: All SQL execution images in this report are original team screenshots extracted from the prior project conversation or supplied PDFs.

Aggregate queries can summarize marketplace activity using functions such as COUNT, AVG and SUM. The verified procedure and trigger demonstrations are documented in Chapter 8.

7.2 UPDATE OPERATION

```
mysql> UPDATE gig
    -> SET view_count = view_count + 10
    -> WHERE gig_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SELECT gig_id, title, view_count
    -> FROM gig
    -> WHERE gig_id = 1;
+--------+-----------------------------------------+------------+
| gig_id | title                                   | view_count |
+--------+-----------------------------------------+------------+
|      1 | Build a React and Node.js Web Application |         20 |
+--------+-----------------------------------------+------------+
1 row in set (0.00 sec)
```

Figure 7.6: UPDATE Operation

The UPDATE increases the view_count for gig_id 1 and the subsequent SELECT confirms the updated value. The exact submitted terminal output is retained.

7.3 DELETE OPERATION

The DELETE operation and its verification are documented below.

```
mysql> INSERT INTO notification
    -> (user_id, notification_type, message, related_id)
    -> VALUES
    -> (1, 'Gig Update', 'Temporary notification for DELETE demonstration.', 1);
Query OK, 1 row affected (0.01 sec)

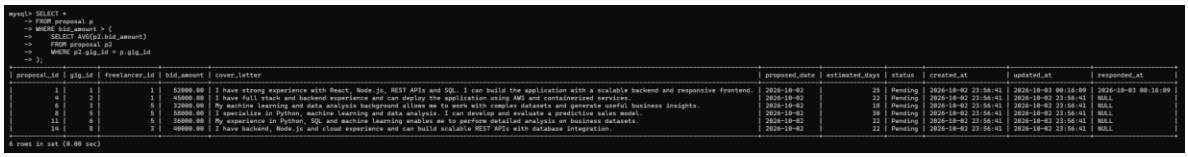
mysql>
mysql> SET @test_id = LAST_INSERT_ID();
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELETE FROM notification
    -> WHERE notification_id = @test_id;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT *
    -> FROM notification
    -> WHERE notification_id = @test_id;
Empty set (0.00 sec)
```

Figure 7.7: DELETE Operation

7.4 CORRELATED QUERY



```
mysql> SELECT *
    -> FROM proposal p
    -> WHERE bid_amount > (
    ->     SELECT AVG(p2.bid_amount)
    ->     FROM proposal p2
    ->     WHERE p2.gig_id != p.gig_id
    -> );
+-------------+--------+---------------+------------+-----------------------
```

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

8.1 STORED PROCEDURES OR FUNCTIONS

MatchFreelancersBySkills(IN input_gig_id INT) joins gig_skill, freelancer_skill, freelancer, and users. It counts matching required skills per freelancer and orders the result by matched-skill count. Verification: CALL MatchFreelancersBySkills(2) returned freelancer_id 3 (Rohan Gupta) with 2 matched skills.



Figure 8.1: Stored Procedure

The verified live database inventory confirmed one stored procedure and one trigger. No stored function was included in that verified object count.

8.2 TRIGGERS

The AFTER INSERT trigger trg_update_freelancer_rating recalculates average_rating and total_reviews for the freelancer associated with the newly inserted review. Dynamic verification used contract 1, freelancer 6, client 2 and rating 5. The values changed from 0.00 / 0 to 5.00 / 1 after INSERT.



Figure 8.2: Trigger Definition

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER trg_update_freelancer_rating
    -> AFTER INSERT ON review
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE freelancer
    ->     SET
    ->         average_rating = (
    ->             SELECT AVG(rating)
    ->             FROM review
    ->             WHERE freelancer_id = NEW.freelancer_id
    ->         ),
    ->         total_reviews = (
    ->             SELECT COUNT(*)
    ->             FROM review
    ->             WHERE freelancer_id = NEW.freelancer_id
    ->         )
    ->     WHERE freelancer_id = NEW.freelancer_id;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

Figure 8.3: CREATE TRIGGER

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO review
    -> (contract_id, client_id, freelancer_id, rating, review_text)
    -> VALUES
    -> (1, 1, 6, 5, 'Excellent work and communication.');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT
    ->     freelancer_id,
    ->     average_rating,
    ->     total_reviews
    -> FROM freelancer
    -> WHERE freelancer_id = 6;
+---------------+----------------+---------------+
| freelancer_id | average_rating | total_reviews |
+---------------+----------------+---------------+
|             6 |           5.00 |             1 |
+---------------+----------------+---------------+
1 row in set (0.00 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

Figure 8.4: Trigger Test and ROLLBACK

8.3 TRANSACTION AND ROLLBACK VERIFICATION

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE proposal
    -> SET
    ->     status = 'Accepted',
    ->     responded_at = CURRENT_TIMESTAMP
    -> WHERE proposal_id = 2
    ->   AND status = 'Pending';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> INSERT INTO contract
    -> (proposal_id, gig_id, freelancer_id, start_date, agreed_amount, status)
    -> SELECT
    ->     proposal_id,
    ->     gig_id,
    ->     freelancer_id,
    ->     CURRENT_DATE,
    ->     bid_amount,
    ->     'Active'
    -> FROM proposal
    -> WHERE proposal_id = 2
    ->   AND status = 'Accepted';
Query OK, 1 row affected (0.00 sec)
Records: 1  Duplicates: 0  Warnings: 0

mysql> UPDATE gig
    -> SET status = 'In Progress'
    -> WHERE gig_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT
    ->     p.proposal_id,
    ->     p.status,
    ->     c.contract_id,
    ->     c.agreed_amount,
    ->     g.status AS gig_status
    -> FROM proposal p
    -> JOIN contract c ON p.proposal_id = c.proposal_id
    -> JOIN gig g ON c.gig_id = g.gig_id
    -> WHERE p.proposal_id = 2;
+-------------+----------+-------------+---------------+-------------+
| proposal_id | status   | contract_id | agreed_amount | gig_status  |
+-------------+----------+-------------+---------------+-------------+
|           2 | Accepted |           3 |      48000.00 | In Progress |
+-------------+----------+-------------+---------------+-------------+
1 row in set (0.00 sec)
```

Figure 8.5: Transaction Verification

The original transaction screenshots show START TRANSACTION, proposal status update, contract insertion, gig status update and COMMIT, followed by SELECT verification of the accepted proposal, created contract and In Progress gig. A separate trigger test is shown above and is rolled back explicitly.

Note on existing data: freelancer 2 had an existing review with rating 5 while the stored aggregate was previously observed as 0.00 / 0. The historical aggregate was not modified. The trigger test verifies new review behavior but does not backfill historical values.

9. FRONT END DEVELOPMENT

9.1 ARCHITECTURE

The Next.js application provides the user interface and server-side API routes. Authenticated requests are handled by Next.js route handlers; MySQL stores authoritative marketplace and transactional records. For semantic matching, Next.js calls a local FastAPI service using a server-side service token. FastAPI generates embeddings and queries persistent ChromaDB storage. The browser does not need direct access to database credentials or the semantic-service token.

9.2 MAIN USER INTERFACES

The frontend includes role-based user experiences for clients and freelancers, gig browsing and details, profile and skill management, proposal workflows, contracts, milestone views, messaging, notifications, and review-related features.

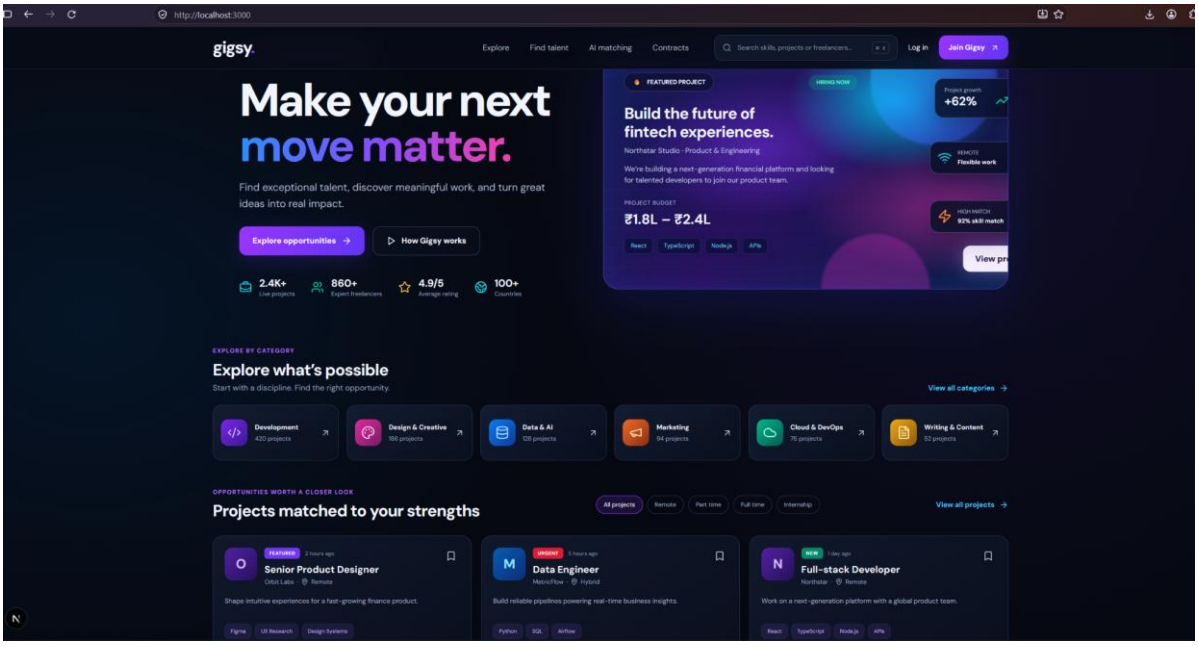


Figure 9.1: Landing Page

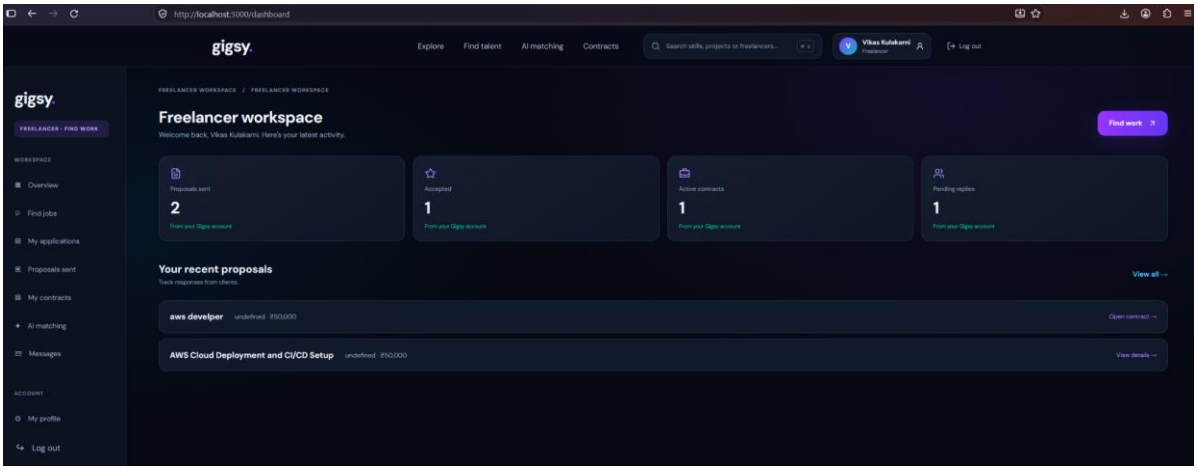


Figure 9.2: Freelancer Dashboard

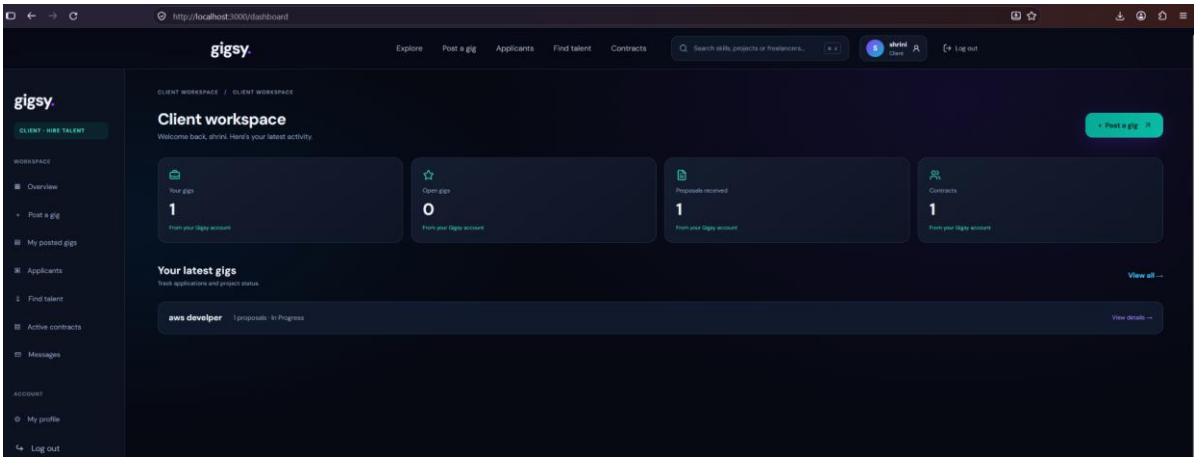


Figure 9.3: Client Dashboard

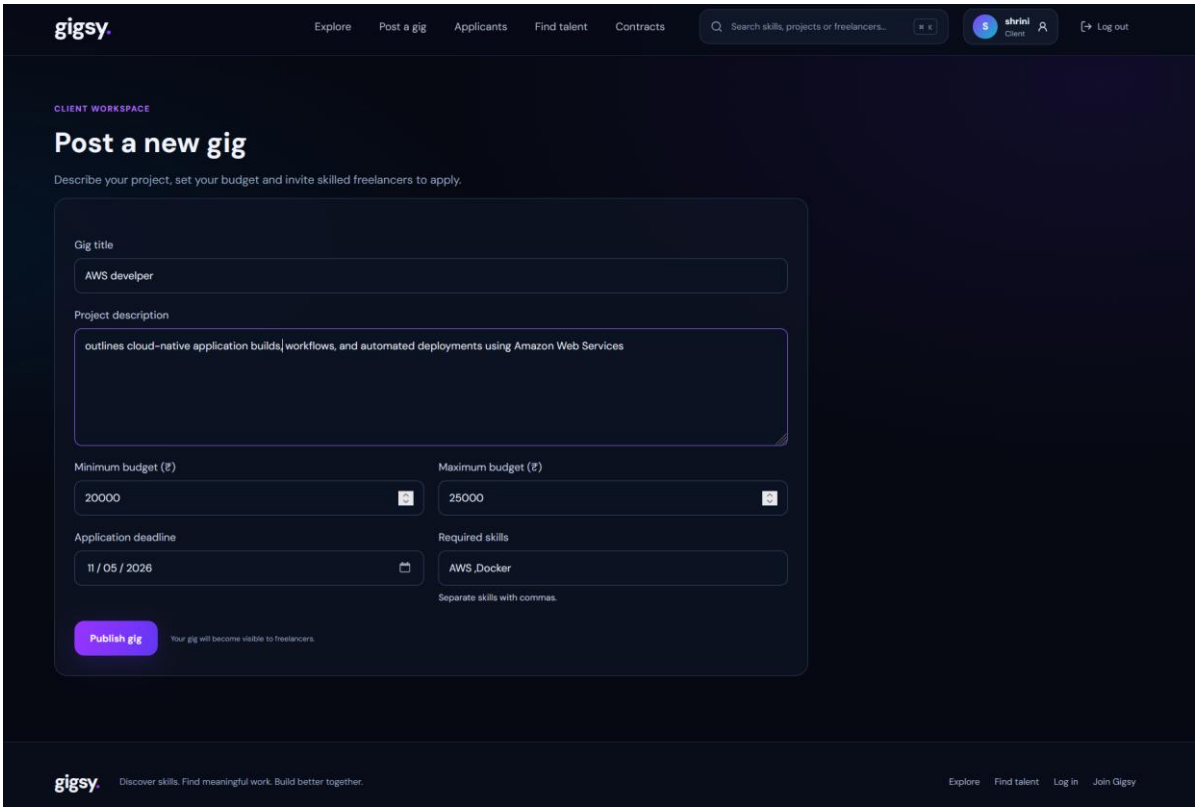


Figure 9.4: Create Gig Form

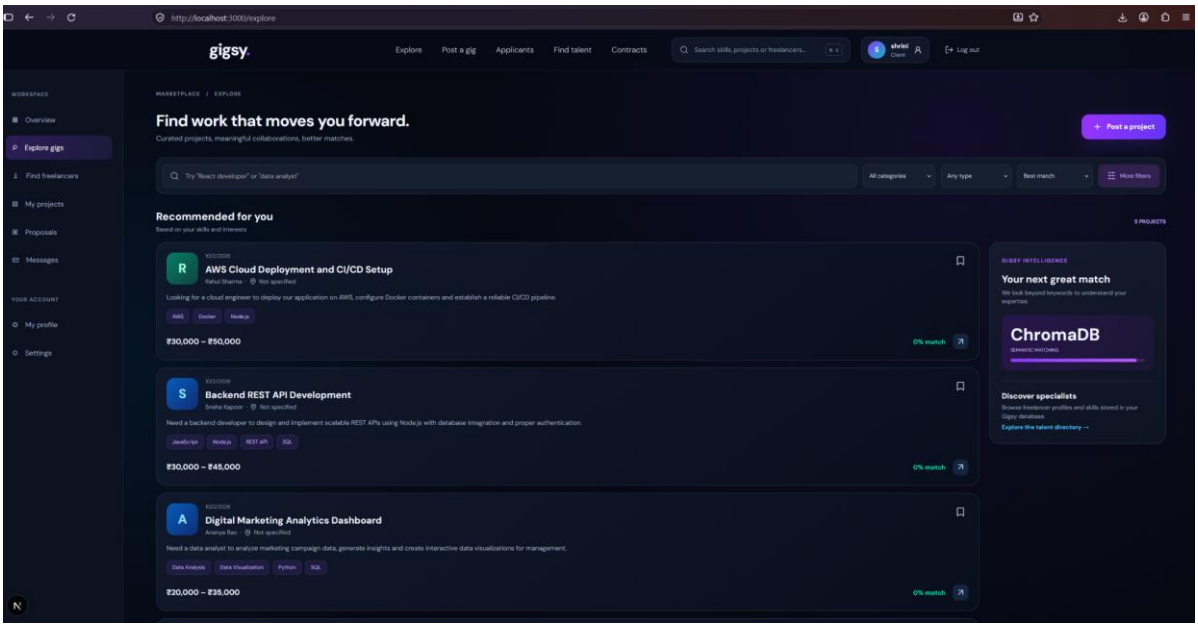


Figure 9.5: Explore Gigs

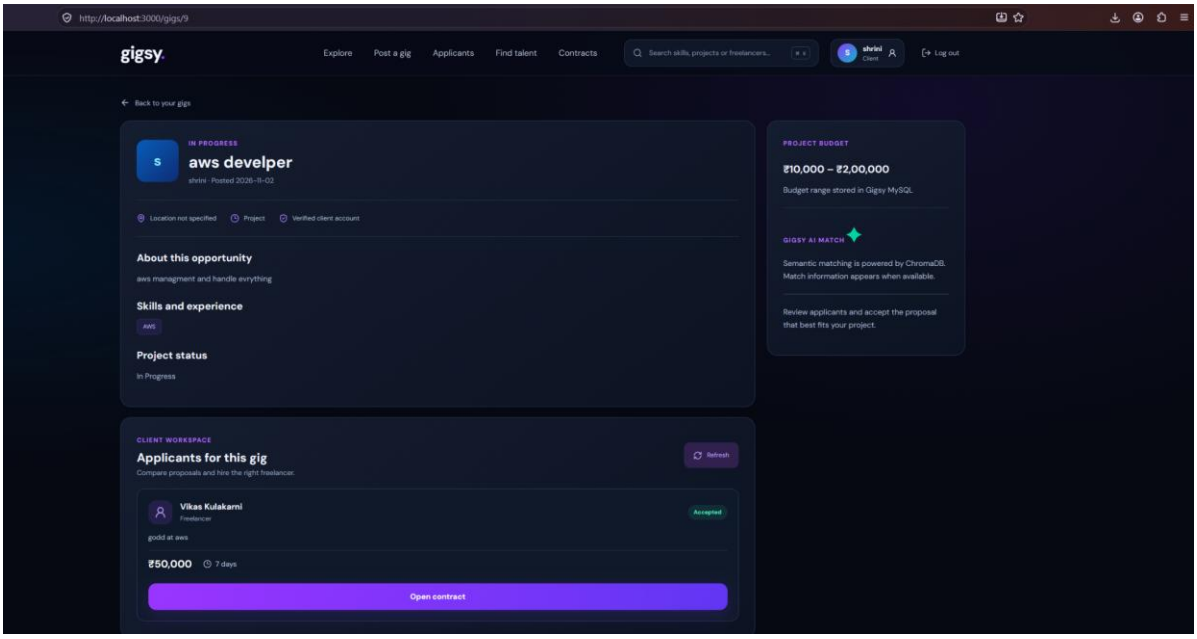


Figure 9.6: Proposal Review

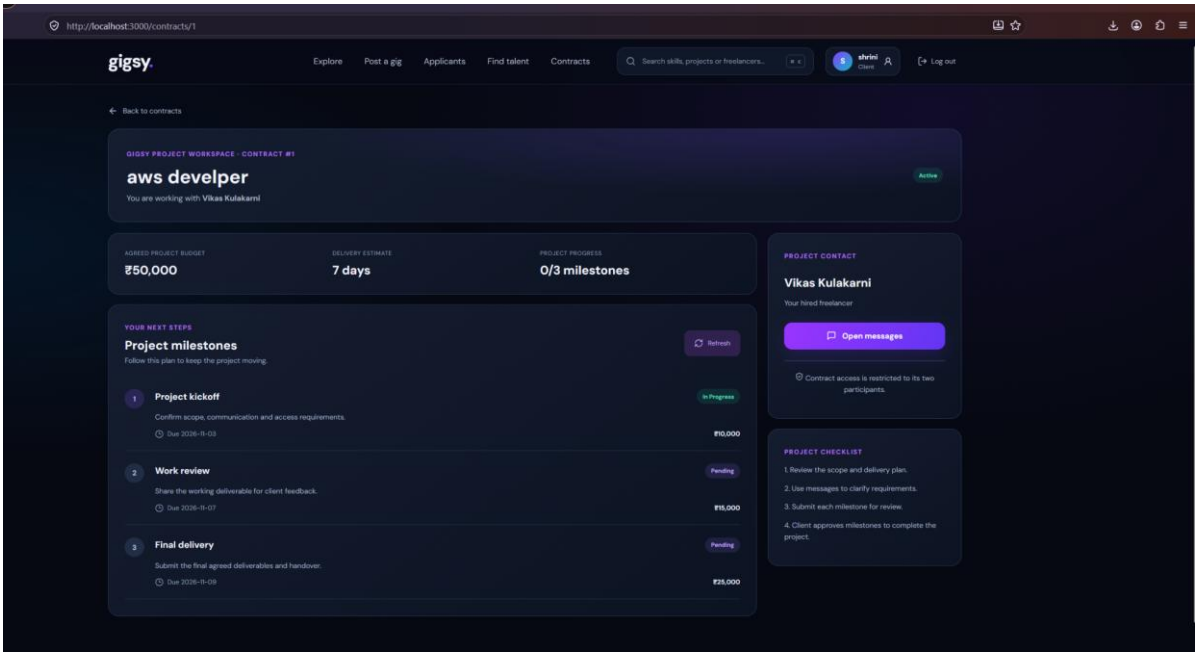


Figure 9.7: Contract and Milestones

9.3 SEMANTIC SEARCH INTEGRATION

Gig descriptions and associated skill names are combined into text documents and embedded with sentence-transformers/all-MiniLM-L6-v2. The model produces 384-dimensional vectors. FastAPI stores vectors, document IDs and metadata (gig_id, title and skills) in persistent ChromaDB collection gigsy_gigs. The collection is configured with cosine distance. During search, the query is embedded with the same model and ChromaDB returns nearest gig IDs and distances; the service converts distance to a

bounded similarity score. Next.js resolves those IDs against MySQL and returns only gigs whose current status is Open. MySQL remains authoritative for gig details and status; ChromaDB provides semantic retrieval.

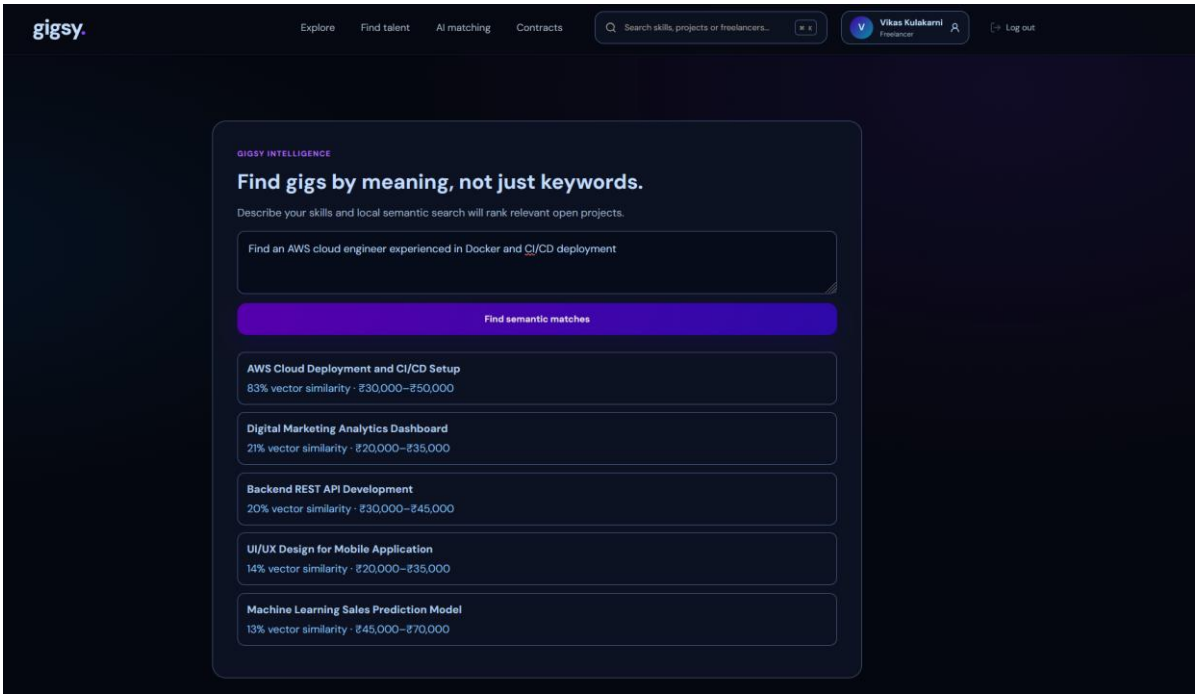


Figure 9.8: Semantic Search Results

Health endpoint: FastAPI health was previously verified, including ChromaDB connectivity, the gigsy_gigs collection, five indexed records and 384-dimensional embeddings. The original health screenshot was not present in the available uploaded evidence, so no substitute image has been inserted.

9.4 API SURFACE

The application includes API routes for gigs, proposals, contracts, and semantic matching. The semantic service exposes health, index, and search endpoints. Protected service routes use the configured service token; administrative indexing uses the indexing secret.

9.5 TESTING AND EVIDENCE

Previously completed validation included npm ci, npx tsc --noEmit, and npm run build. The Next.js health endpoint returned a successful database connection, and FastAPI health reported the ChromaDB collection and embedding model. Semantic search was confirmed working in the application.

9.6 DATABASE TECHNOLOGY JUSTIFICATION

MySQL is used for structured marketplace entities and operations requiring relational constraints, joins and transactional consistency, including users, gigs, proposals, contracts and milestones. ChromaDB is used for unstructured gig descriptions and skill text, where semantic similarity is more useful than exact keyword matching. MySQL remains the source of truth, while ChromaDB provides candidate retrieval.

9.7 SECURITY AND DATA INTEGRITY

Passwords are hashed with scrypt before storage. Session tokens are signed and placed in HTTP-only cookies with SameSite protection. SQL statements use parameterized queries through mysql2. Administrative indexing is protected by an indexing secret, and Next.js communicates with the semantic service using a separate server-side token. Database credentials and tokens must be configured through environment variables and must not be committed. Primary keys, declared foreign keys, unique constraints and rating checks provide database-level integrity. Some current gigsy_* workflow references are application-enforced because the supplied schema export does not declare foreign keys for those tables.

9.8 TEST CASES AND VERIFICATION SUMMARY

Table 9.1 summarizes verification evidence available during project preparation. “Previously verified” refers to results recorded during project work; it does not mean tests were re-executed while preparing this document. The DDL/DML screenshot panels in Chapters 5 and 6 are the original panels retained from the supplied PDF; they are not newly generated or represented as fresh captures.

Table 9.1: Test cases and available verification evidence.

Test ID	Test / Expected behaviour	Recorded result	Status
TC-01	MySQL health endpoint connects to configured database.	Successful database connection previously reported.	Previously verified
TC-02	Client/Freelancer registration validates inputs and creates role-specific profiles.	Route implements role validation, password hashing and transactional profile creation; live re-run not recorded here.	Source reviewed
TC-03	Login rejects invalid credentials and creates a signed HTTP-only session for valid credentials.	Login route uses password verification and signed session cookies; live re-run not recorded here.	Source reviewed
TC-04	Gig listing supports status, text and skill filtering.	Parameterized query and filtering route inspected in source.	Source reviewed

Test ID	Test / Expected behaviour	Recorded result	Status
TC-05	Proposal submission rejects invalid role, gig and bid values.	Route enforces authenticated Freelancer role, validates input and checks gig status.	Source reviewed
TC-06	Matching procedure ranks freelancers by matching required skills.	CALL with gig ID 2 returned freelancer ID 3 (Rohan Gupta) with two matched skills.	Previously verified
TC-07	Review trigger recalculates average rating and review count after INSERT.	Values changed from 0.00/0 to 5.00/1; ROLLBACK restored 0.00/0 and removed test review.	Previously verified
TC-08	Contract milestone amounts reconcile with contract amount.	Recorded contract amount ₹50,000 and three milestones totaling ₹50,000.	Previously verified
TC-09	ChromaDB health reports collection and embedding dimensions.	Report records gigsy_gigs, five indexed records and 384-dimensional embeddings.	Previously verified
TC-10	Semantic search returns relevant open gigs.	Successful search recorded in report; API source filters results against current Open gigs.	Previously verified / source reviewed
TC-11	TypeScript checks and production build complete.	npm ci, npx tsc --noEmit and npm run build previously reported successful.	Previously verified

9.9 LIMITATIONS AND FUTURE ENHANCEMENTS

The current schema contains legacy marketplace relations alongside newer gigsy_* workflow relations. Some references in the latter are application-enforced rather than declared foreign keys. The semantic service is configured for local development and requires the Python service to run separately. Future work includes adding database-level foreign keys after a safe data audit, reconciling legacy and current workflow models, scheduled vector re-indexing, richer ranking signals, deployment with persistent managed storage, and expanded automated integration and security tests.

10. CONCLUSION AND TEAM CONTRIBUTIONS

10.1 CONCLUSION

Gigsy combines a relational freelance marketplace with vector-based semantic retrieval. MySQL manages structured accounts, gigs and workflow records, while ChromaDB stores embeddings derived from gig descriptions and skills. The Next.js application integrates these services through API routes and a FastAPI semantic layer. The project demonstrates relational modelling, constraints, views, stored procedures, triggers, transaction handling and semantic similarity search. Original SQL execution screenshots are included in Chapters 5–8.

10.2 TEAM PARTICIPATION

The project team comprises Vikas Kulakarni (PES2UG24CS922), Shrinivas Shrikant Jainapur (PES2UG24CS494) and Shivangouda (PES2UG24CS471). Vikas Kulakarni served as project lead and primary integrator, with the largest share of end-to-end engineering, architecture, Next.js/API integration, authentication, MySQL workflow integration, ChromaDB/FastAPI semantic-search integration, debugging, validation and final report coordination. Shrinivas contributed to relational modelling and SQL/DML evidence. Shivangouda contributed to marketplace workflow review, application testing and supporting evidence. All members should confirm that this allocation accurately reflects their work.

10.3 INDIVIDUAL CONTRIBUTION DETAILS

Table 10.1: Team members, responsibilities and contributions.

Team member	Primary responsibilities	Contribution summary
Vikas Kulakarni PES2UG24CS922	• Project leadership • problem definition and requirements interpretation • end-to-end system architecture • Next.js frontend/API and backend integration • MySQL connection and workflow implementation • authentication and authorization • FastAPI–ChromaDB semantic-search integration • embedding/indexing pipeline • debugging and integration testing • GitHub coordination • technical documentation and final report assembly	Lead contributor and primary integrator. Owned the largest share of implementation and cross-component work, coordinated the database, web application and semantic-search services, resolved integration issues, and consolidated the final technical documentation and submission evidence.

Team member	Primary responsibilities	Contribution summary
Shrinivas Shrikant Jainapur PES2UG24CS494	• Relational database design support • ER diagram and relational schema preparation/review • SQL table and relationship work • DML query preparation and verification • support for advanced SQL demonstrations and database evidence	Contributed to database modelling and SQL/DML evidence, including query demonstrations and review of relational design.
Shivangouda PES2UG24CS471	• Marketplace workflow and interface support • client/freelancer flow review • application testing • review of UI behaviour • support for project evidence collection and presentation	Contributed to application workflow validation, interface review, functional testing and supporting project documentation.

11. REQUIREMENT COVERAGE AND FINALIZATION STATUS

11.1 EVIDENCE STATUS

Final evidence status: Original MySQL screenshots are included for database object inventory, table structures, views, INSERT, UPDATE, DELETE, SELECT, JOIN, aggregate functions, GROUP BY, HAVING, nested and correlated subqueries, transactions, trigger creation/testing and stored procedure creation/execution. The integrated Gigsy schema is documented in Chapter 5.

Evidence note: the original DDL/ER/schema screenshots and the complete DML/advanced-SQL execution screenshot sequence supplied by the team are included in Chapters 5–8. No screenshots were generated or reconstructed. The supplied DDL evidence shows SHOW TABLES and DESCRIBE output; the advanced SQL evidence also shows CREATE TRIGGER, CREATE PROCEDURE, CREATE VIEW and ALTER TABLE ADD FULLTEXT INDEX. A DROP execution screenshot is not present in the supplied PDFs.

11.2 REQUIREMENT COVERAGE MATRIX

Table 11.1: Faculty requirement coverage and remaining actions.

Faculty requirement	Coverage in Gigsy	Status / remaining action
Real-world problem, objectives and scope	Chapters 1–2 define the freelance matching problem, objectives, scope and functional/non-functional requirements.	Documented
SQL plus NoSQL/vector database	MySQL stores structured marketplace data; ChromaDB stores text embeddings for semantic retrieval.	Implemented and documented
ER diagram and relational schema	Chapter 3 contains the ER model; Chapter 4 maps the 17 base tables and declared relationships.	Documented; verify diagram against final schema export
DDL: CREATE, ALTER, DROP, keys, constraints, indexes and views	Chapter 5 includes current SHOW TABLES, DESCRIBE and view outputs. The schema dump contains DROP TABLE IF EXISTS statements.	DDL evidence included; standalone DROP execution screenshot not located
DML: INSERT, UPDATE, DELETE and SELECT	Chapter 6 documents the original INSERT operation. Chapter 7 covers SELECT, JOIN, aggregates, GROUP BY, HAVING, UPDATE, DELETE, nested and correlated queries.	Execution evidence included
Four or more advanced DB concepts	Transactions, views, stored procedure, trigger and vector similarity search are documented.	Implemented/documented; demonstrate during viva

Faculty requirement	Coverage in Gigsy	Status / remaining action
Vector model, storage, metric and retrieval	Semantic-search section documents all-MiniLM-L6-v2, 384-dimensional embeddings, ChromaDB collection, cosine distance and MySQL revalidation.	Documented
Next.js web interface and database integration	Application chapters describe the Next.js interface, API routes, MySQL integration and FastAPI semantic service.	Implemented; run locally for demonstration
Authentication with at least two roles	Client and Freelancer roles are described with registration/login and role-aware workflows.	Implemented/documented
Validation, integrity and security	Report describes parameterized SQL, password hashing, signed HTTP-only sessions, environment-based secrets and integrity controls.	Documented; retain only controls present in final code
Git/GitHub README and commits	Project repository and setup documentation are referenced.	Confirm repository visibility, latest commits and README before submission
Testing, demonstration and individual contribution	Chapter 9 records available verification evidence; Section 10.3 records team contributions.	Run final demo and confirm contribution allocation with all members

REFERENCES

[1] MySQL 8.0 Reference Manual. Oracle. https://dev.mysql.com/doc/

[2] Next.js Documentation. Vercel. https://nextjs.org/docs

[3] FastAPI Documentation. https://fastapi.tiangolo.com/

[4] Chroma Documentation. https://docs.trychroma.com/

[5] Sentence Transformers Documentation. https://sbert.net/

GitHub Link: https://github.com/vikaskulakarni/Gigsy

APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Term	Definition
API	Application Programming Interface – a defined way for one software component to request services from another.
Cardinality	The number of instances of one entity that can be associated with instances of another entity in an ER model.
ChromaDB	An open-source vector database used here to store and query gig embeddings.
Cosine distance	A similarity measure between two vectors based on the angle between them; smaller values mean more similar text.
DBMS	Database Management System – software used to define, store, query and manage data.
DDL	Data Definition Language – SQL statements such as CREATE, ALTER and DROP that define database structure.
DML	Data Manipulation Language – SQL statements such as INSERT, UPDATE, DELETE and SELECT that work with stored data.
Embedding	A fixed-length numeric vector that represents the meaning of a piece of text.
ER Model	Entity–Relationship model – a conceptual design showing entities, attributes and relationships.
FastAPI	A Python web framework used for the semantic search service.
FK	Foreign Key – a column that references the primary key of another table to enforce referential integrity.
Gig	A job or project posted by a client on the Gigsy marketplace.
MySQL	The relational database management system used as the source of truth for Gigsy.
Next.js	A React-based framework used for the Gigsy user interface and server-side API routes.
PK	Primary Key – a column or set of columns that uniquely identifies each row of a table.
Semantic search	Search that matches the meaning of a query rather than exact keywords, using vector similarity.
SQL	Structured Query Language – the standard language for relational databases.
Stored procedure	A named, precompiled set of SQL statements stored in the database and run with CALL.
Transaction	A group of SQL operations executed as one unit that is either committed or rolled back as a whole.
Trigger	A stored routine that runs automatically when a specified event, such as an INSERT, occurs on a table.
UI	User Interface.

Term	Definition
View	A saved SELECT query that behaves like a virtual table.